

```

for event in pygame.event.get():
    if (event.type == pygame.QUIT) or \
        ((event.type == pygame.KEYDOWN) and
         (event.key == pygame.K_ESCAPE)):
        # Tell the current scene we're leaving
        self.oCurrentScene.leave()
        pygame.quit()
        sys.exit()

    eventsList.append(event)

# Here, we let the current scene process all events,
# do any "per frame" actions in its update method,
# and draw everything that needs to be drawn.
❸ self.oCurrentScene.handleInputs(eventsList, keysDownList)
❹ self.oCurrentScene.update()
❺ self.oCurrentScene.draw()

# 11 - Update the window
❻ pygame.display.update()

# 12 - Slow things down a bit
clock.tick(self.framesPerSecond)

```

Listing 15-7: The `run()` method of the `SceneMgr` class

The `run()` method is the key to how the scene manager works. Remember that all scenes must be polymorphic—at a minimum, each must implement a `handleInputs()` and a `draw()` method. Each time through the loop, the `run()` method does the following:

- Gets a list of all keyboard keys ❶ (`False` means up, `True` means down).
- Builds a list of events ❷ that have happened since the last time through the loop.
- Makes calls to the polymorphic methods ❸ of the current scene. The current scene is always kept in an instance variable named `self.oCurrentScene`. In the call to the scene's `handleInputs()` method, the scene manager passes in the list of events that have happened and the list of keys. Each scene is responsible for handling the events and for dealing with the state of the keyboard.